UNIT V: Virtual Functions, Polymorphism & Exception Handling

Question Bank Answers: Debugging & Pseudo-Code Formulation

Name	Nikitha R
Reg. No	19252421
Course Code & Name	DSA0105 Object Oriented Programming with C++

Contents

- **Section A: Debugging** (5 problems): line numbers and defect category, vtable / stack-unwinding theory, corrected code and dry run with console output.
- **Section B: Pseudo-Code** (5 problems): design overview, structured pseudo-code (ABSTRACT CLASS / OVERRIDE METHOD / TRY / CATCH notation) and trace tables.
- Every corrected C++ program was compiled with g++ (`-Wall -Wextra`) and run; the outputs shown are the real outputs. The buggy originals were compiled and run too, so the quoted behavior is real.

SECTION A: DEBUGGING QUESTIONS

Question 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

Topic: Virtual Functions, Virtual Destructors, Dynamic Dispatch

Step 1: Analyse the problem statement

- **Expected:** deleting through `Shape*` destroys the whole `ComplexPolygon` including its 16,000-byte vertex array (1000 vertices × 2 doubles × 8 bytes).
- **Actual** (confirmed by running the buggy program): the output ends with "Shape destroyed."; the line "Deallocating polygon vertex heap memory." never appears.

Step 2: Error found

Line	Classification	Defect
Line 7: `~Shape()` (not virtual)	Runtime Bug: Undefined Behavior / Memory Leak	Deleting a derived object through a base pointer when the base destructor is non-virtual is undefined behavior by the C++ standard. In practice, only `~Shape()` runs.

Step 3: Root cause explanation (Tasks 1 to 3)

- **Static (early) binding.** For a non-virtual function, the compiler chooses the function from the *declared type of the pointer* at compile time. `delete shapePtr` with `Shape *shapePtr` is compiled as a direct call to `Shape::~Shape()` followed by freeing the memory. `~ComplexPolygon()` is never called, so `delete[] vertexArray` is skipped and the vertex array leaks (and the memory is released with the wrong size information).
- **Dynamic (late) binding mechanics.** If at least one function in a class is virtual, the compiler gives the class a **vtable**: a hidden static table of function pointers, one per virtual function. Every object of the class carries a hidden **vptr** to its class's vtable, set by the constructor. A virtual call `p->f()` is compiled as: load `p->vptr`, index to the slot for `f`, call through that pointer. The target is therefore decided from the **object's real type** at run time (one extra indirection per call, plus one pointer of space per object).
- **With a virtual destructor** `~Shape()` becomes a vtable slot. `delete shapePtr` calls `~ComplexPolygon()` through the vptr: the derived destructor body runs first (freeing the vertex array); when it ends, the compiler-generated epilogue calls the base destructor `~Shape()`. Destruction order is therefore the reverse of construction: derived first, base last.
- **Rule.** Any class that is meant to be used polymorphically (has virtual functions or is deleted through a base pointer) must have a **virtual destructor**. Mark overriding destructors with `override` for compiler checking.

Step 4: Corrected code

```
#include <iostream>

class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }

    virtual ~Shape() {                          // virtual destructor
        std::cout << "Shape destroyed." << std::endl;
    }

    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double *vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount << " vertices on heap." << std::endl;
    }

    ~ComplexPolygon() override {
        std::cout << "Deallocating polygon vertex heap memory." << std::endl;
        delete[] vertexArray;
    }

    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount << " vertices." << std::endl;
    }
};

int main() {
    Shape *shapePtr = new ComplexPolygon(1000);
    shapePtr->render();

    delete shapePtr;                            // derived dtor runs, then base dtor
    return 0;
}
```

Step 5: Dry run and verified output

Order	Event	Printed
1	new ComplexPolygon(1000): base constructor first	Shape created.
2	derived constructor allocates 2000 doubles	Allocated 1000 vertices on heap.
3	shapePtr->render(): virtual, resolves to the override	Rendering polygon with 1000 vertices.
4	delete shapePtr: vptr leads to ~ComplexPolygon, which frees the array	Deallocating polygon vertex heap memory.
5	then the base destructor runs automatically	Shape destroyed.

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Question 2: Unreachable Exception Handler (Base Before Derived)

Topic: Order of Catch Handlers, Custom Exception Hierarchy

Step 1: Analyse the problem statement

- **Expected:** a withdrawal of 750 from a balance of 500 reports the shortfall of $250.
- **Actual:** compiles (GCC only warns that the derived handler "will be caught by earlier handler") and prints "General Banking Error: Account overdrawn!"; the shortfall is never shown (confirmed by running the buggy program).

Step 2: Error found

Line	Classification	Defect
Line 25: `catch (const BankingException &be)` placed before the derived handler on Line 28	Logic Defect (unreachable handler)	The base-class handler matches every `InsufficientFundsException`, so the more specific handler below it can never run.

Step 3: Root cause explanation (Tasks 1 to 3)

- **How handlers are selected.** When an exception is thrown, the runtime compares its type with the handlers of the enclosing try block **strictly in the order written, from top to bottom**, and runs the *first* one that matches. A handler for type `T&` matches an exception of class `T` *or any class derived from T* (an *is-a* match, since a derived object can be bound to a base reference). `InsufficientFundsException` is a `BankingException`, so the first handler wins. There is no "best match" search as in overload resolution.
- **Rule.** Always order handlers **most derived first, base class next,** `catch (...)` **last**. The compiler requires the catch-all to be last ("'...' handler must be the last handler for its try block" is an error) and only warns about unreachable class handlers.
- **Catch by const reference.** Catching by value would copy and slice the object (losing derived data such as `shortfall`) and invoke the copy constructor; the reference keeps the original object and its dynamic type. Give the base class a virtual destructor/virtual `getMessage()` so derived overrides are used.

Step 4: Corrected code (three handlers: specific, base, catch-all)

```
#include <iostream>
#include <string>

class BankingException {
protected:
    std::string msg;
public:
    BankingException(std::string m) : msg(m) {}
    virtual ~BankingException() {}
    virtual std::string getMessage() const { return msg; }
};

class InsufficientFundsException : public BankingException {
private:
    double shortfall;
public:
    InsufficientFundsException(std::string m, double sf)
        : BankingException(m), shortfall(sf) {}

    double getShortfall() const { return shortfall; }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException("Account overdrawn!", amount - balance);
    }
}

int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException &ife) {      // most derived FIRST
        std::cout << ife.getMessage() << " Shortfall of: $" << ife.getShortfall() << std::endl;
    }
    catch (const BankingException &be) {                 // then the base class
        std::cout << "General Banking Error: " << be.getMessage() << std::endl;
    }
    catch (...) {                                        // universal fallback LAST
        std::cout << "Unknown error occurred." << std::endl;
    }
    return 0;
}
```

Step 5: Dry run and verified output

Step	Event	State
1	processWithdrawal(500, 750): 750 > 500, so throw InsufficientFundsException("Account overdrawn!", 250)	stack unwinds out of the function
2	handler 1: InsufficientFundsException exact match	executes; handlers 2 and 3 are skipped
3	prints message and shortfall (750 - 500)	Account overdrawn! Shortfall of: $250

```
Account overdrawn! Shortfall of: $250
```

Question 3: Object Slicing from throw e; vs. throw; Rethrow

Topic: Rethrowing Exceptions, Catching by Reference, Object Slicing

Step 1: Analyse the problem statement

- **Expected:** the client's `TimeoutError` handler runs and prints the 5000 ms duration.
- **Actual** (confirmed by running): the audit line prints, then "Client caught generic DatabaseError: ..." appears; the TimeoutError handler is skipped.

Step 2: Error found

Line	Classification	Defect
Line 25: `throw e;`	Logic Defect (Object Slicing)	Throws a *new* exception object copy-initialized from the static type of `e`, which is `const DatabaseError&`. The `TimeoutError` part (`timeoutMs`) is sliced away.

Step 3: Root cause explanation (Tasks 1 and 2)

- **Throw expression semantics.** `throw expr;` creates a fresh exception object by copy-initializing it from *expr*, using the **static type** of the expression. Since `e` is declared as a `DatabaseError` reference, the thrown object is a plain `DatabaseError` (the base subobject copy), even though the original object was a `TimeoutError`. Its dynamic type is now DatabaseError, so `catch (const TimeoutError&)` in main cannot match, and the next handler, `catch (const DatabaseError&)`, does. The message survives (it lives in the base) but `timeoutMs` is lost.
- **Bare `throw;`.** A throw expression with no operand re-throws the **currently handled exception object itself**, without copying it. The original object, with its full dynamic type `TimeoutError`, propagates up the stack, so the specific handler matches and `timeoutMs` is intact. It is also cheaper (no copy) and is only valid inside a handler (otherwise `std::terminate` is called).
- **Rule of thumb.** Catch by (const) reference, and rethrow with `throw;`, never `throw e;`.

Step 4: Corrected code

```
#include <iostream>
#include <exception>
#include <string>

class DatabaseError : public std::exception {
protected:
    std::string errorDetails;
public:
    DatabaseError(std::string msg) : errorDetails(msg) {}
    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;
    TimeoutError(std::string msg, int ms) : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError("Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError &e) {
        std::cout << "[AUDIT LOG]: Intercepted DB error." << std::endl;
        throw;                                    // rethrow the ORIGINAL object, no copy
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError &te) {
        std::cout << "Client caught TimeoutError! Duration: " << te.timeoutMs << " ms" << std::endl;
    }
    catch (const DatabaseError &de) {
        std::cout << "Client caught generic DatabaseError: " << de.what() << std::endl;
    }
    return 0;
}
```

Step 5: Dry run and verified output

Step	Event	Dynamic type of the exception in flight
1	executeQuery() throws TimeoutError("...shard 3", 5000)	TimeoutError
2	intermediateAuditor catches it as const DatabaseError& (reference: no slicing) and logs	TimeoutError (same object)
3	throw; rethrows the same object	TimeoutError
4	main: first handler TimeoutError matches	TimeoutError

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

Question 4: Unimplemented Pure Virtual Function Leaves a Class Abstract

Topic: Pure Virtual Functions, Abstract Classes

Step 1: Analyse the problem statement

- **Expected:** `VideoPacket packet(1042);` creates a usable packet object.
- **Actual:** compile-time error.

Step 2: Error found

Line	Classification	Defect
Line 24: `VideoPacket packet(1042);`	Compilation Error	GCC: **"cannot declare variable 'packet' to be of abstract type 'VideoPacket'"**, with the note "because the following virtual functions are pure within 'VideoPacket': virtual int NetworkPacket::getPacketSize()".

Step 3: Root cause explanation (Tasks 1 and 2)

- **What makes a class abstract.** A class with at least one **pure virtual function** (`= 0`) is abstract: its vtable has a slot with no implementation, so no complete object could ever be safely used. Pure virtual functions define an *interface contract*. The language forbids creating objects of abstract classes (only pointers/references to them are allowed).
- **Inherited purity.** A derived class inherits every pure virtual function. `VideoPacket` overrides `serialize()` but not `getPacketSize()`, so `getPacketSize()` is *still pure* in VideoPacket and VideoPacket is itself abstract. A derived class becomes **concrete** only when it provides overrides for **all** inherited pure virtual functions.
- **Fix.** Implement `int getPacketSize() override` in `VideoPacket`. Then `VideoPacket packet(1042);` is legal, and a `NetworkPacket*` can point to it for polymorphic use.
- **Note.** Using `override` is good practice: it makes the compiler verify that the signature really matches a base virtual function.

Step 4: Corrected code

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() {}

    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: " << frameID << std::endl;
    }

    int getPacketSize() override {               // the missing override
        return 1500;
    }
};

int main() {
    VideoPacket packet(1042);                     // concrete now
    packet.serialize();
    std::cout << "Packet size: " << packet.getPacketSize() << " bytes" << std::endl;

    NetworkPacket *p = &packet;                   // polymorphic use through the interface
    std::cout << "Via base pointer: " << p->getPacketSize() << " bytes" << std::endl;
    return 0;
}
```

Step 5: Dry run and verified output

Step	Action	Result
1	VideoPacket packet(1042): all pure functions overridden, so the class is concrete	object created
2	packet.serialize()	Serializing H.264 Video Frame: 1042
3	packet.getPacketSize()	1500
4	NetworkPacket *p = &packet; p->getPacketSize(): dynamic dispatch	1500

```
Serializing H.264 Video Frame: 1042
Packet size: 1500 bytes
Via base pointer: 1500 bytes
```

Question 5: Object Slicing in Pass-by-Value Polymorphism

Topic: Runtime Polymorphism, Reference/Pointer Binding, Slicing

Step 1: Analyse the problem statement

- **Expected:** `triggerEncounter(boss)` prints "Dragon breathes scorching fire!".
- **Actual** (confirmed by running): it prints "Entity performs generic scratch." even though `attack()` is virtual and overridden.

Step 2: Error found

Line	Classification	Defect
Line 20: `void triggerEncounter(Entity e)`	Logic Defect (Object Slicing)	By-value parameter: only the `Entity` part of `boss` is copied into `e`.

Step 3: Root cause explanation (Tasks 1 and 2)

- **Slicing.** The parameter `e` is a *complete, independent* `Entity` object, created by `Entity`'s copy constructor from the `Dragon`. The copy constructor builds a true Entity: it sets the new object's **vptr to the Entity vtable**. The Dragon-specific members and the Dragon vptr are not part of the copy.
- **Why the virtual call goes to Entity.** `e.attack()` does use the vptr, but the vptr of `e` points to `Entity`'s vtable, whose `attack` slot holds `Entity::attack`. The call is virtual, but the object it is virtual *on* is no longer a Dragon.
- **Polymorphism needs indirection.** Dynamic dispatch works only through a **pointer or a reference** to the original object, because these refer to the existing Dragon, with its own vptr, instead of building a new object.
- **Fix.** Take the parameter by reference (`const Entity &e`) or by pointer (`const Entity *e`). A const reference is preferred: no copy, no slicing, and the `const` member function `attack() const` is callable on it.

Step 4: Corrected code

```
#include <iostream>

class Entity {
public:
    virtual ~Entity() {}
    virtual void attack() const {
        std::cout << "Entity performs generic scratch." << std::endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        std::cout << "Dragon breathes scorching fire!" << std::endl;
    }
};

// Take the polymorphic parameter by const reference: no copy, no slicing
void triggerEncounter(const Entity &e) {
    e.attack();                                  // resolved through the vptr -> Dragon::attack
}

int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

Step 5: Dry run and verified output

Version	Parameter	Object seen by e.attack()	vptr points to	Output
Buggy	`Entity e`	new Entity copy (slice)	Entity vtable	Entity performs generic scratch.
Fixed	`const Entity &e`	the original Dragon	Dragon vtable	Dragon breathes scorching fire!

```
Dragon breathes scorching fire!
```

SECTION B: PSEUDO-CODE WRITING QUESTIONS

Question 1: Universal Payment Gateway with Runtime Polymorphism

Topic: Pure Virtual Functions, Runtime Polymorphism, Abstract Base Classes

Design overview

- **ABSTRACT CLASS PaymentGateway** declares the contract: PURE VIRTUAL METHOD Authorize(amount) and Capture(), plus a VIRTUAL DESTRUCTOR so polymorphic deletion is safe.
- **CreditCardGateway:** authorizes only if the card number has exactly 16 digits, the CVV has exactly 3 digits and the amount is positive.
- **CryptoGateway:** authorizes only if the wallet hash starts with "0x" and the wallet balance covers the amount plus a simulated gas fee. *Assumed for the simulation:* gas fee = 0.2% of the amount; wallet balance $2,000.
- **Late binding:** Main stores the chosen gateway in `POINTER TO PaymentGateway`; the same calls `gatewayPtr->Authorize(1500.00)` and `gatewayPtr->Capture()` run the derived implementation selected at run time.

Structured pseudo-code

```
ABSTRACT CLASS PaymentGateway
  PUBLIC:
    PURE VIRTUAL METHOD Authorize(REAL amount) RETURNS BOOLEAN
    PURE VIRTUAL METHOD Capture() RETURNS BOOLEAN
    VIRTUAL DESTRUCTOR()
      DISPLAY "PaymentGateway released"
    ENDDESTRUCTOR
ENDCLASS

CLASS CreditCardGateway EXTENDS PaymentGateway
  PRIVATE:
    STRING cardNumber
    STRING cvv
    REAL   pendingAmount
    BOOLEAN authorized
  PUBLIC:
    CONSTRUCTOR(STRING number, STRING code)
      cardNumber <- number ; cvv <- code
      pendingAmount <- 0.0 ; authorized <- FALSE
    ENDCONSTRUCTOR

    OVERRIDE METHOD Authorize(REAL amount) RETURNS BOOLEAN
      IF LENGTH(cardNumber) != 16 OR NOT ALL_DIGITS(cardNumber) THEN
        DISPLAY "Card declined: number must be 16 digits"
        RETURN FALSE
      ENDIF
      IF LENGTH(cvv) != 3 OR NOT ALL_DIGITS(cvv) THEN
        DISPLAY "Card declined: CVV must be 3 digits"
        RETURN FALSE
      ENDIF
      IF amount <= 0 THEN
        RETURN FALSE
      ENDIF
      pendingAmount <- amount
      authorized    <- TRUE
      RETURN TRUE
    ENDMETHOD

    OVERRIDE METHOD Capture() RETURNS BOOLEAN
      IF authorized = FALSE THEN
        RETURN FALSE
      ENDIF
      DISPLAY "Card charged: ", pendingAmount
      authorized <- FALSE
      RETURN TRUE
    ENDMETHOD

    OVERRIDE DESTRUCTOR()
```

```
      DISPLAY "CreditCardGateway released"
    ENDDESTRUCTOR
ENDCLASS

CLASS CryptoGateway EXTENDS PaymentGateway
  PRIVATE:
    STRING  walletHash
    REAL    walletBalance
    REAL    pendingTotal
    BOOLEAN authorized
  PUBLIC:
    CONSTRUCTOR(STRING hash, REAL balance)
      walletHash <- hash ; walletBalance <- balance
      pendingTotal <- 0.0 ; authorized <- FALSE
    ENDCONSTRUCTOR

    OVERRIDE METHOD Authorize(REAL amount) RETURNS BOOLEAN
      REAL gasFee, requiredFunds
      IF SUBSTRING(walletHash, 0, 2) != "0x" THEN
        DISPLAY "Wallet rejected: hash must start with 0x"
        RETURN FALSE
      ENDIF
      gasFee        <- amount * 0.002                    // simulated gas fee (0.2%)
      requiredFunds <- amount + gasFee
      IF amount <= 0 OR requiredFunds > walletBalance THEN
        DISPLAY "Declined: insufficient funds for amount + gas"
        RETURN FALSE
      ENDIF
      pendingTotal <- requiredFunds
      authorized   <- TRUE
      RETURN TRUE
    ENDMETHOD

    OVERRIDE METHOD Capture() RETURNS BOOLEAN
      IF authorized = FALSE THEN
        RETURN FALSE
      ENDIF
      walletBalance <- walletBalance - pendingTotal
      authorized    <- FALSE
      DISPLAY "Crypto transfer complete, balance: ", walletBalance
      RETURN TRUE
    ENDMETHOD

    OVERRIDE DESTRUCTOR()
      DISPLAY "CryptoGateway released"
    ENDDESTRUCTOR
ENDCLASS
```

```
BEGIN Main
  DECLARE gatewayPtr AS POINTER TO PaymentGateway
  STRING choice
  BOOLEAN ok
  READ choice                                          // "CARD" or "CRYPTO"
  IF choice = "CRYPTO" THEN
    gatewayPtr <- NEW CryptoGateway("0xA1B2C3", 2000.0)
  ELSE
    gatewayPtr <- NEW CreditCardGateway("4111111111111111", "123")
  ENDIF
  ok <- gatewayPtr->Authorize(1500.00)              // late binding
  IF ok = TRUE THEN
    ok <- gatewayPtr->Capture()                     // late binding
  ENDIF
  DELETE gatewayPtr                                 // virtual destructor chain
END Main
```

Trace: $1,500 cryptocurrency payment (choice = "CRYPTO")

Step	Statement	Dispatch decided by	Calculation / effect	Result
1	gatewayPtr <- NEW CryptoGateway("0xA1B2C3", 2000.0)	upcast to PaymentGateway*	vptr of object = CryptoGateway vtable	pointer holds a Crypto object
2	gatewayPtr->Authorize(1500.00)	vptr to vtable slot Authorize, resolves to CryptoGateway::Authorize	prefix "0x" ok; gasFee = 1500 × 0.002 = 3.00; required = 1503.00 <= 2000.00	TRUE; authorized, pendingTotal = 1503.00
3	gatewayPtr->Capture()	vtable slot Capture, resolves to CryptoGateway::Capture	walletBalance = 2000.00 - 1503.00 = 497.00	TRUE; prints balance 497
4	DELETE gatewayPtr	virtual destructor: vptr gives ~CryptoGateway, then base	derived destructor runs first, then ~PaymentGateway	prints "CryptoGateway released", "PaymentGateway released"

Had the user chosen "CARD", the same two lines in Main would have executed the CreditCardGateway overrides without any change to Main: that is runtime polymorphism.

Question 2: Resilient Banking Engine with Nested Try-Catch & Rethrowing

Topic: Custom Exception Hierarchy, Nested try-catch, Rethrowing

Design overview

- **Hierarchy:** TransactionException (reason) is the base; AccountFrozenException and DailyLimitExceededException (adds excessAmount) derive from it.
- **Three tiers:** DebitLedger (throws), AuditTransactionWrapper (catches by reference, writes the audit log, then **RETHROW**), Main (specific handlers).
- **RETHROW** (bare) forwards the original exception object, so the derived type survives and Main's specific handler matches (no slicing).
- **Handler order in Main:** most derived first, then TransactionException, then CATCH (...) as the final fallback.

Structured pseudo-code

```
CLASS TransactionException
  PROTECTED:
    STRING reason
  PUBLIC:
    CONSTRUCTOR(STRING r)
      reason <- r
    ENDCONSTRUCTOR
    VIRTUAL DESTRUCTOR()
    METHOD GetReason() RETURNS STRING
      RETURN reason
    ENDMETHOD
ENDCLASS

CLASS AccountFrozenException EXTENDS TransactionException
  PUBLIC:
    CONSTRUCTOR() : TransactionException("Account is frozen")
    ENDCONSTRUCTOR
ENDCLASS

CLASS DailyLimitExceededException EXTENDS TransactionException
  PRIVATE:
    REAL excessAmount
  PUBLIC:
    CONSTRUCTOR(REAL excess) : TransactionException("Daily limit exceeded")
      excessAmount <- excess
    ENDCONSTRUCTOR
    METHOD GetExcess() RETURNS REAL
      RETURN excessAmount
    ENDMETHOD
ENDCLASS

CONSTANT REAL DAILY_LIMIT <- 50000.0

FUNCTION DebitLedger(BOOLEAN accountFrozen, REAL withdrawnToday, REAL amount)
  IF accountFrozen = TRUE THEN
    THROW NEW AccountFrozenException()
  ENDIF
  IF withdrawnToday + amount > DAILY_LIMIT THEN
    THROW NEW DailyLimitExceededException(withdrawnToday + amount - DAILY_LIMIT)
  ENDIF
  DISPLAY "Ledger debited: ", amount
ENDFUNCTION

FUNCTION AuditTransactionWrapper(BOOLEAN accountFrozen, REAL withdrawnToday, REAL amount)
  TRY
    DebitLedger(accountFrozen, withdrawnToday, amount)
  CATCH (e AS TransactionException)                // by reference: no slicing
    DISPLAY "[SECURITY AUDIT] Transaction failed: ", e.GetReason()
```

```
    RETHROW                                          // same object continues upward
  ENDTRY
ENDFUNCTION

BEGIN Main
  TRY
    AuditTransactionWrapper(TRUE, 0.0, 1000.0)      // frozen account, $1,000 withdrawal
    DISPLAY "Withdrawal complete"
  CATCH (e AS AccountFrozenException)
    DISPLAY "Notice: withdrawal blocked, account frozen. Contact your branch."
  CATCH (e AS DailyLimitExceededException)
    DISPLAY "Notice: daily limit exceeded by $", e.GetExcess()
  CATCH (e AS TransactionException)
    DISPLAY "Notice: transaction failed: ", e.GetReason()
  CATCH (...)
    DISPLAY "Notice: unexpected system error"
  ENDTRY
  DISPLAY "Session continues"
END Main
```

Dry-run trace: $1,000 withdrawal on a frozen account

Step	Tier / location	Event	Control transfer	Output
1	Main, TRY block entered	calls AuditTransactionWrapper(TRUE, 0, 1000)	normal call	-
2	AuditTransactionWrapper, TRY entered	calls DebitLedger(...)	normal call	-
3	DebitLedger	accountFrozen = TRUE, so THROW NEW AccountFrozenException()	normal flow stops; stack unwinding of DebitLedger's frame begins	-
4	AuditTransactionWrapper CATCH (e AS TransactionException)	exception is an AccountFrozenException; base-class handler matches (is-a)	control jumps to handler	[SECURITY AUDIT] Transaction failed: Account is frozen
5	AuditTransactionWrapper handler	RETHROW: same object, still dynamic type AccountFrozenException	wrapper frame unwinds, exception propagates to caller	-
6	Main handlers tested in order	1st: AccountFrozenException matches (exact)	handler runs; other handlers skipped	Notice: withdrawal blocked, account frozen. Contact your branch.
7	Main after ENDTRY	normal execution resumes (the "Withdrawal complete" line was skipped)	-	Session continues

Question 3: Polymorphic Game Engine with Virtual Destructors

Topic: Virtual Functions, Virtual Destructors, Polymorphic Object Lifecycles

Design overview

- **ABSTRACT CLASS GameObject** with PURE VIRTUAL METHOD Update(deltaTime) and Render(), and a VIRTUAL DESTRUCTOR.
- **ParticleSystem** owns a heap array of coordinates (2 values per particle). Its destructor releases the array; because the base destructor is virtual, DEALLOCATE through a GameObject pointer runs ~ParticleSystem first and then ~GameObject.
- **Driver:** array of base pointers, one simulation step (Update + Render for every entity), then cleanup in a loop.

Structured pseudo-code

```
ABSTRACT CLASS GameObject
  PUBLIC:
    PURE VIRTUAL METHOD Update(REAL deltaTime)
    PURE VIRTUAL METHOD Render()
    VIRTUAL DESTRUCTOR()
      DISPLAY "~GameObject: base resources released"
    ENDDESTRUCTOR
ENDCLASS

CLASS ParticleSystem EXTENDS GameObject
  PRIVATE:
    POINTER TO REAL particleCoords                // x0,y0,x1,y1,... on the heap
    INTEGER particleCount
  PUBLIC:
    CONSTRUCTOR(INTEGER count)
      INTEGER i
      particleCount  <- count
      particleCoords <- ALLOCATE ARRAY[count * 2] OF REAL
      FOR i <- 0 TO count * 2 - 1 STEP 1
        particleCoords[i] <- 0.0
      ENDFOR
    ENDCONSTRUCTOR

    OVERRIDE METHOD Update(REAL deltaTime)
      INTEGER i
      FOR i <- 0 TO particleCount - 1 STEP 1
        particleCoords[2*i + 1] <- particleCoords[2*i + 1] - 9.8 * deltaTime   // gravity on y
      ENDFOR
    ENDMETHOD

    OVERRIDE METHOD Render()
      DISPLAY "Rendering ", particleCount, " particles"
    ENDMETHOD

    OVERRIDE DESTRUCTOR()
      DISPLAY "~ParticleSystem: releasing particle array (", particleCount, " particles)"
      DEALLOCATE particleCoords
    ENDDESTRUCTOR
ENDCLASS

BEGIN Main
  DECLARE sceneEntities AS ARRAY[2] OF POINTER TO GameObject
  INTEGER i
  sceneEntities[0] <- NEW ParticleSystem(100)
  sceneEntities[1] <- NEW ParticleSystem(50)

  FOR i <- 0 TO 1 STEP 1                          // one simulation step
    sceneEntities[i]->Update(0.016)
```

```
    sceneEntities[i]->Render()
  ENDFOR

  FOR i <- 0 TO 1 STEP 1                          // cleanup phase
    DEALLOCATE sceneEntities[i]
  ENDFOR
END Main
```

Trace of the cleanup phase (destructor order for each DEALLOCATE sceneEntities[i])

Step	Operation	Mechanism	Printed / effect
1	DEALLOCATE sceneEntities[0] (static type GameObject*, real type ParticleSystem(100))	destructor is virtual, so the call goes through the vptr	most derived destructor selected
2	~ParticleSystem body executes first	derived destructor runs before base	~ParticleSystem: releasing particle array (100 particles); heap array of 200 REALs freed
3	derived destructor ends, compiler-inserted call to ~GameObject	bases destroyed after the derived part, reverse of construction	~GameObject: base resources released
4	object memory returned to the heap	-	entity 0 fully gone
5	DEALLOCATE sceneEntities[1] (ParticleSystem(50)): same sequence	-	~ParticleSystem (50 particles; 100 REALs freed), then ~GameObject

Without the virtual keyword on ~GameObject, steps 2 and 5 would be skipped: both particle arrays would leak (Section A, Question 1).

Question 4: File Stream Pipeline with Stack Unwinding & Catch-All

Topic: Stack Unwinding, Resource Acquisition Is Initialization (RAII), Catch-All Handler

Design overview

- **RAII:** ResourceGuard acquires the handle in its CONSTRUCTOR and releases it in its DESTRUCTOR. Because the guard is a *local automatic object* in IngestDataStream, cleanup is guaranteed on every exit path (normal return or exception) without any explicit close call.
- **Stack unwinding:** when THROW executes, the runtime leaves each active function frame in turn and destroys its local objects *before* control reaches the matching CATCH. Hence the handle is released before the handler's first line runs.
- Main uses a specific CATCH for CRCValidationException (reads expected and calculated checksums) and a final CATCH (...) for anything unforeseen.

Structured pseudo-code

```
CLASS ResourceGuard
  PRIVATE:
    INTEGER handle
  PUBLIC:
    CONSTRUCTOR(STRING fileName)
      handle <- AcquireHandle(fileName)
      DISPLAY "Handle acquired for ", fileName
    ENDCONSTRUCTOR
    DESTRUCTOR()
      ReleaseHandle(handle)
      DISPLAY "Handle released (destructor)"
    ENDDESTRUCTOR
    METHOD ReadBlock() RETURNS ARRAY OF BYTE
ENDCLASS

CLASS CRCValidationException
  PRIVATE:
    INTEGER expectedCRC
    INTEGER calculatedCRC
  PUBLIC:
    CONSTRUCTOR(INTEGER expected, INTEGER calculated)
      expectedCRC <- expected ; calculatedCRC <- calculated
    ENDCONSTRUCTOR
    METHOD GetExpected() RETURNS INTEGER
      RETURN expectedCRC
    ENDMETHOD
    METHOD GetCalculated() RETURNS INTEGER
      RETURN calculatedCRC
    ENDMETHOD
ENDCLASS

FUNCTION IngestDataStream(STRING fileName)
  ResourceGuard guard(fileName)                    // local automatic object (RAII)
  INTEGER calculatedCRC, expectedCRC
  ARRAY OF BYTE block
  calculatedCRC <- 0
  WHILE NOT EndOfFile(guard) DO
    block <- guard.ReadBlock()
    calculatedCRC <- UpdateCRC(calculatedCRC, block)
  ENDWHILE
  expectedCRC <- ReadStoredCRC(guard)
  IF calculatedCRC != expectedCRC THEN
    THROW NEW CRCValidationException(expectedCRC, calculatedCRC)
  ENDIF
  DISPLAY "Stream validated"
ENDFUNCTION                                        // guard destroyed here on normal exit

BEGIN Main
```

```
  TRY
    DISPLAY "Starting ingestion"
    IngestDataStream("telemetry.dat")
    DISPLAY "Ingestion finished"
  CATCH (e AS CRCValidationException)
    DISPLAY "CRC failure: expected 0x", HEX(e.GetExpected()), ", got 0x", HEX(e.GetCalculated())
  CATCH (...)
    DISPLAY "Unknown ingestion error"
  ENDTRY
  DISPLAY "Pipeline terminated safely"
END Main
```

Trace: stack unwinding with a CRC mismatch (expected 0xA1B2, calculated 0x9F00)

Step	Location / stack	Event	Output
1	Main	TRY entered	Starting ingestion
2	Main calls IngestDataStream; frame pushed	local object guard constructed (acquires handle)	Handle acquired for telemetry.dat
3	IngestDataStream	blocks read, CRC computed = 0x9F00; stored CRC = 0xA1B2	-
4	IngestDataStream	0x9F00 != 0xA1B2: THROW NEW CRCValidationException(0xA1B2, 0x9F00). Normal flow stops; "Stream validated" is skipped	-
5	stack unwinding: IngestDataStream's frame is left	destructor of guard runs automatically: handle released	Handle released (destructor)
6	Main: handlers searched in order	CATCH (e AS CRCValidationException) matches; CATCH (...) is not reached	-
7	handler body runs (after the handle is already released)	reads expected/calculated from the exception object	CRC failure: expected 0xA1B2, got 0x9F00
8	after ENDTRY	"Ingestion finished" was skipped	Pipeline terminated safely

Question 5: Scientific Telemetry Calibration with noexcept Contracts

Topic: Specifying Exceptions, noexcept Contracts, Boundary Assertions

Design overview

- **CalibrateThermalSensors** is potentially throwing (NOEXCEPT(FALSE)): it converts voltage to temperature and THROWs OutOfBoundsException when the result lies outside -273.15 to +1500.0 degrees C. *Assumed conversion for the simulation:* temperature = rawVoltage × 500.0 - 273.15.
- **CommitTelemetry** is NOEXCEPT: a contract that it never lets an exception escape. If one did, the runtime would call `std::terminate` (abort). To honor the contract it wraps its whole body in TRY with CATCH (...) that suppresses and records the failure.
- **Main** forms the safety harness: only the throwing function sits in the TRY; the non-throwing commit is called only after a valid calibration.

Structured pseudo-code

```
CONSTANT REAL MIN_TEMP <- -273.15
CONSTANT REAL MAX_TEMP <- 1500.0

CLASS OutOfBoundsException
  PRIVATE:
    REAL offendingValue
  PUBLIC:
    CONSTRUCTOR(REAL v)
      offendingValue <- v
    ENDCONSTRUCTOR
    METHOD GetValue() RETURNS REAL
      RETURN offendingValue
    ENDMETHOD
ENDCLASS

FUNCTION CalibrateThermalSensors(REAL rawVoltage) RETURNS REAL  NOEXCEPT(FALSE)
  REAL temperature
  temperature <- (rawVoltage * 500.0) - 273.15
  IF temperature < MIN_TEMP OR temperature > MAX_TEMP THEN
    THROW NEW OutOfBoundsException(temperature)    // explicit throwing rule
  ENDIF
  RETURN temperature
ENDFUNCTION

FUNCTION CommitTelemetry(REAL finalTemp)  NOEXCEPT
  TRY
    WriteToAvionicsBus(finalTemp)                  // may fail internally
    DISPLAY "Committed to bus: ", finalTemp
  CATCH (...)                                      // suppress EVERYTHING
    LogLocalFault("bus write failed")              // record, never propagate
  ENDTRY
ENDFUNCTION                                        // contract kept: nothing escapes

BEGIN Main
  REAL rawVoltage, finalTemp
  READ rawVoltage
  TRY
    finalTemp <- CalibrateThermalSensors(rawVoltage)
    CommitTelemetry(finalTemp)                     // skipped when calibration throws
  CATCH (e AS OutOfBoundsException)
    DISPLAY "Calibration rejected: reading ", e.GetValue(), " is outside physical limits"
  CATCH (...)
    DISPLAY "Unexpected calibration failure"
  ENDTRY
END Main
```

Trace: a valid reading and an invalid reading

Case	rawVoltage	Calibrate computation	Bounds check	Flow	Output
Valid	1.2	1.2 × 500.0 - 273.15 = 326.85	-273.15 <= 326.85 <= 1500.0 passes	returns 326.85; CommitTelemetry runs, its inner TRY completes normally	Committed to bus: 326.85
Invalid (fault)	-0.0537	-0.0537 × 500.0 - 273.15 = -300.00	-300.00 < -273.15 violates the lower bound	THROW NEW OutOfBounds Exception(-300.00); CommitTelemetry is never called; Main's CATCH (e AS OutOfBoundsException) runs	Calibration rejected: reading -300 is outside physical limits

Why the contract matters: if CommitTelemetry let a bus-write exception escape, the NOEXCEPT promise would be broken and the system would abort (std::terminate) instead of degrading gracefully. The internal CATCH (...) converts any failure into a local log entry, so the avionics process keeps running.